

Network Traffic Discrimination: VoIP vs. Non-VoIP Classification

Alexandru-Gabriel Lefterache, Catalina Costache

PICSC

Contents

1	Introduction	3
2	Methodology	3
2.1	Network Traffic Dataset	4
2.2	Flow Aggregation and Identification	4
2.3	Feature Extraction	5
2.3.1	Flow Representation	5
2.3.2	Directional Volumetric Features	7
2.3.3	Flow Rate Features	7
2.3.4	Inter-Arrival Time Features	8
2.3.5	Higher-Order Distribution Statistics	8
2.4	Feature Engineering	10
2.5	Class Imbalance Mitigation	11
2.6	Model Selection and Hyperparameter Optimization	14
2.6.1	Random Forest Classifier	14
2.6.2	Support Vector Machine Classifier	14
2.6.3	Multi-Layer Perceptron Classifier	15
3	Experimental Design	16
3.1	Flow Filtering Configurations	16
3.2	Data Split	18
4	Results and Analysis	18
5	Conclusion	25

1 Introduction

The network traffic discrimination task aims to automatically distinguish *Voice over IP* (VoIP) traffic from non-VoIP traffic within packet-switched networks. This task is critical for enabling quality-of-service (QoS) provisioning, traffic prioritization, congestion control, network security, and lawful interception in modern IP networks where multiple applications share the same infrastructure.

In this task, each network flow, which represents a packet sequence, is analyzed and classified into one of two categories: VoIP traffic, which carries real-time voice communications, and non-VoIP traffic, which includes data generated by web browsing, file transfers, streaming services, and other network applications. Unlike traditional port-based or payload-inspection approaches, modern traffic discrimination methods rely solely on **statistical and temporal characteristics** of packet streams, allowing them to operate effectively even when traffic is encrypted.

VoIP traffic exhibits distinctive behavioral patterns due to its real-time and interactive nature. These patterns typically include small and fixed packet sizes, regular inter-packet arrival times, bidirectional flows, and low tolerance for latency and jitter. In contrast, non-VoIP traffic often demonstrates more variable packet sizes, bursty transmission behavior, and higher tolerance to delay, reflecting the characteristics of applications such as HTTP, FTP, or video streaming.

The discrimination task is commonly formulated as a binary classification problem, where features are extracted from packet headers and timing information over fixed time windows or per flow. Typical features include packet size statistics, inter-arrival time distributions, packet rate, flow duration, and directional traffic ratios. These features are then used to train machine learning or deep learning models that can learn the underlying differences between VoIP and non-VoIP traffic patterns.

Accurate VoIP traffic discrimination enables network operators to efficiently allocate resources, prioritize delay-sensitive voice traffic, detect anomalous behavior such as VoIP tunneling or misuse, and improve overall network performance. As encryption and dynamic port usage become increasingly prevalent, behavior-based traffic discrimination remains a key technique for maintaining visibility and control in modern communication networks.

2 Methodology

This section presents the methodological framework for VoIP traffic discrimination, encompassing data collection, flow aggregation, feature extraction, dimensionality reduction, and class imbalance mitigation strategies. The approach emphasizes behavioral traffic characterization through statistical and temporal features extracted from packet-level observations.

2.1 Network Traffic Dataset

VPN-nonVPN dataset (ISCXVPN2016) [1]

A diverse corpus of network traffic was collected to evaluate binary classification performance between VoIP and non-VoIP traffic classes. The dataset comprises audio streams from real-time communication applications including Skype and Google Hangouts, alongside general network traffic encompassing HTTP web browsing, peer-to-peer file sharing, video streaming, FTP transfers, and background network activity.

Raw packet captures were subjected to preprocessing to ensure data quality and analytical feasibility. Packets lacking essential header information, specifically those without TCP or UDP transport layer headers, were excluded from analysis. Following established practices in traffic classification literature [?], flows containing only a single packet were removed, as inter-arrival time features—critical for behavioral discrimination cannot be computed from singleton observations. Flows with missing or malformed attributes were either discarded or subjected to imputation depending on the completeness of available information.

This preprocessing yields a cleaned dataset suitable for flow-level aggregation and feature extraction, ensuring that subsequent analysis operates on complete and well-formed network flows.

2.2 Flow Aggregation and Identification

Network traffic was aggregated into flows to capture application-level behavioral patterns. Unlike packet-level analysis, flow-level characterization enables the extraction of temporal and statistical features that reflect the sustained communication patterns inherent to different application types.

Bidirectional Flow Definition

Network flows were defined using the standard five-tuple:

$$\text{FlowKey} = (\text{IP}_{\text{src}}, \text{IP}_{\text{dst}}, \text{Port}_{\text{src}}, \text{Port}_{\text{dst}}, \text{Protocol}), \quad (1)$$

which uniquely identifies a unidirectional stream between two endpoints. To capture bidirectional conversations, important for interactive traffic such as VoIP, flow keys were normalized lexicographically by IP addresses:

$$\text{FlowKey}_{\text{norm}} = \begin{cases} (\text{IP}_{\text{src}}, \text{IP}_{\text{dst}}, \text{Port}_{\text{src}}, \text{Port}_{\text{dst}}, \text{Protocol}) & \text{if } \text{IP}_{\text{src}} < \text{IP}_{\text{dst}} \\ (\text{IP}_{\text{dst}}, \text{IP}_{\text{src}}, \text{Port}_{\text{dst}}, \text{Port}_{\text{src}}, \text{Protocol}) & \text{otherwise} \end{cases} \quad (2)$$

This ensures that packets traveling in either direction are assigned to the same flow, enabling the extraction of bidirectional features. Each flow was then assigned a compact numerical identifier for efficient tracking and indexing.

Flow Extraction

Inspired by the **ISCX/CICFlowMeter** [1] method, network flows were constructed using configurable temporal aggregation windows to investigate the effect of time resolution on classification performance. Three window sizes were evaluated: **15 seconds**, **120 seconds**, and **600 seconds**. As suggested by the authors, shorter windows tend to improve performance by capturing fine-grained temporal dynamics, though they may fragment longer conversations, whereas longer windows provide more complete flow representations but can dilute discriminative temporal patterns due to extended periods of inactivity.

Each flow is uniquely identified by its flow key and maintains a list of packets, the initial direction, and the start timestamp. Flows are terminated either when the **maximum allowed duration is exceeded** or upon **receiving TCP FIN/RST flags**. Any **remaining active flows are flushed** at the end of the aggregation and also **flows containing only a single packet were excluded** from the outset.

Flow Statistics Analysis

The distribution of flows across applications is highly imbalanced, as shown in Figure 1. *Skype Chat* and *Email* dominate the dataset, each representing roughly 20% of all flows, followed by VOIP applications. Streaming traffic (e.g., Netflix, Spotify, Vimeo, YouTube) accounts for about 15%, Video Calls for 9%, File Transfers for 7%, and other CHAT applications around 8%. Several minor classes contribute less than 2% individually. This skewed distribution suggests that class imbalance strategies, such as stratification or resampling, are important for modeling.

Mean packet counts per class further highlight traffic differences (Figure 2). Chat and email flows have very few packets per flow (typically below 20), reflecting short exchanges. File transfer flows show extremely high packet counts (thousands of packets), consistent with large uploads/downloads. Streaming flows exhibit moderate counts, VoIP flows have medium counts due to frequent small packets, and Video Calls have higher counts than VoIP but remain below file transfers. These patterns indicate that total packet count is a strong discriminative feature for traffic classification.

2.3 Feature Extraction

The feature extraction pipeline transforms raw packet sequences into structured feature vectors capturing temporal, volumetric, and distributional characteristics of network flows. The extraction process operates on complete flow objects and generates **29 features** organized into complementary feature families.

2.3.1 Flow Representation

Each network flow is represented as a structured object containing metadata and packet-level observations:

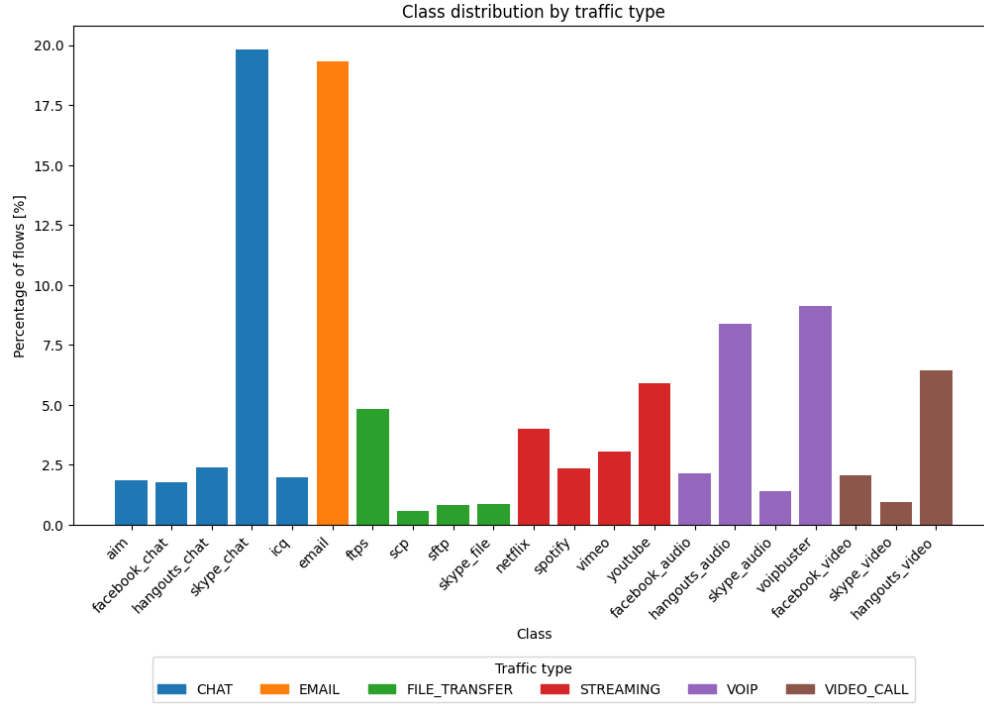


Figure 1: Distribution of network flows by class. Bars are colored by traffic type. Computed for 15s flows.

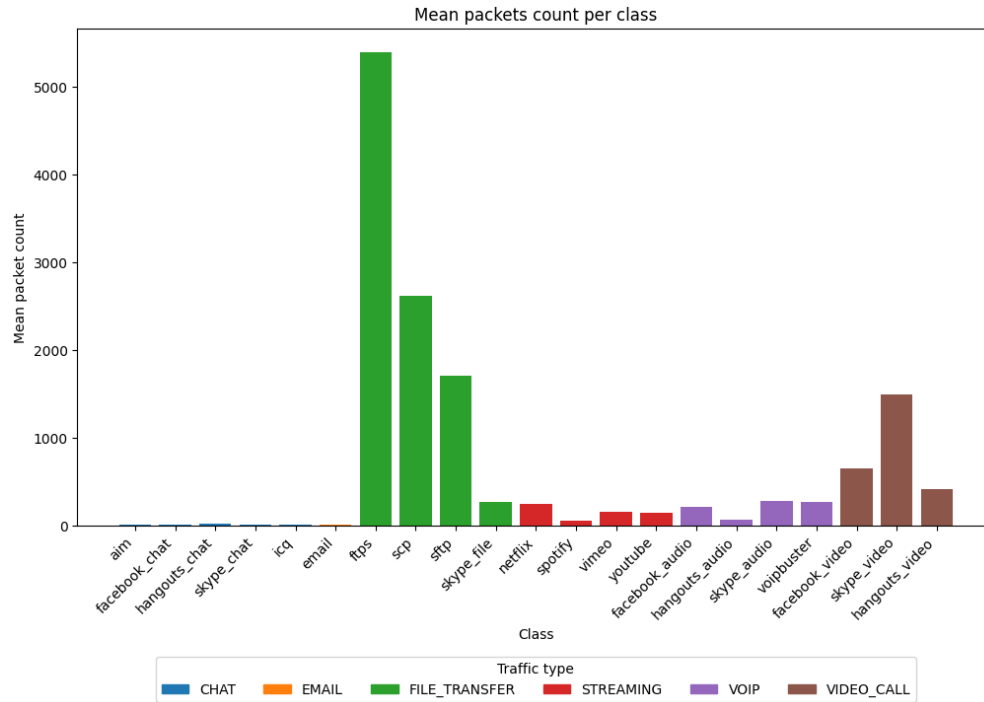


Figure 2: Mean packet count per class. Bars are colored by traffic type. Computed for 15s flows.

$$\text{Flow} = \{\text{flow_key}, \text{init_dir}, \text{packets}, \text{duration}, \text{protocol}, \text{total_bytes}, \text{packet_count}\} \quad (3)$$

where **flow_key** is the normalized five-tuple identifier, **init_dir** indicates the flow initiator, **packets** is a chronologically ordered sequence of packet observations with timestamps and sizes, **duration** represents total flow duration in seconds, and **protocol** specifies the transport layer protocol (TCP or UDP).

Flow Metadata Features

Two flow-level metadata features are directly incorporated into the feature vector.

- **Flow duration (T):** Elapsed time between the first and last packet in the flow.
- **Transport protocol:** Encoded categorical indicator (TCP or UDP).

2.3.2 Directional Volumetric Features

Packets are partitioned into forward and backward directions based on the flow initiator. This partitioning enables the extraction of direction-aware volumetric statistics.

Packet Counts

$$N_{\text{fwd}} = |P_{\text{fwd}}| \quad (4)$$

$$N_{\text{bwd}} = |P_{\text{bwd}}| \quad (5)$$

Mean Packet Lengths

$$\bar{L}_{\text{fwd}} = \frac{1}{N_{\text{fwd}}} \sum_{p \in P_{\text{fwd}}} L(p), \quad N_{\text{fwd}} > 0 \quad (6)$$

$$\bar{L}_{\text{bwd}} = \frac{1}{N_{\text{bwd}}} \sum_{p \in P_{\text{bwd}}} L(p), \quad N_{\text{bwd}} > 0 \quad (7)$$

2.3.3 Flow Rate Features

Transmission intensity is captured using flow-level rate metrics:

$$\text{Flow Bytes/sec} = \frac{B_{\text{total}}}{T}, \quad \text{Flow Packets/sec} = \frac{N_{\text{total}}}{T}, \quad T > 0 \quad (8)$$

where B_{total} is the total number of transmitted bytes and N_{total} is the total packet count.

2.3.4 Inter-Arrival Time Features

Inter-arrival times (IATs) characterize packet timing behavior and are extracted for three complementary sequences:

- **FIAT (Forward IAT)**: Inter-arrival times between consecutive packets in the forward direction.
- **BIAT (Backward IAT)**: Inter-arrival times between consecutive packets in the backward direction.
- **FlowIAT**: Inter-arrival times across all packets in the flow, independent of direction.

Forward IAT Statistics For the forward direction, six statistical features were computed to characterize temporal behavior: **total**, **mean**, **median**, **minimum**, **maximum**, and **standard deviation**. These features help capture regularity and periodicity in packet transmission, typical of VoIP flows.

Backward IAT Statistics Analogous statistics (**total**, **mean**, **median**, **minimum**, **maximum**, **standard deviation**) were extracted for backward traffic. Comparing forward and backward IATs can reveal directional asymmetries in packet timing.

Flow-Level IAT Statistics Five aggregate statistics (**mean**, **median**, **minimum**, **maximum**, **standard deviation**) summarize overall flow temporal behavior. VoIP traffic is typically characterized by low mean and standard deviation of FlowIAT, reflecting codec-specific packetization intervals (e.g., 20 ms for G.711), while other traffic types like web browsing show higher variability with bursty patterns.

2.3.5 Higher-Order Distribution Statistics

Skewness Skewness quantifies the asymmetry of probability distributions:

$$\gamma_1(x) = \frac{\frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^3}{\left[\frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2 \right]^{3/2}} \quad (9)$$

Two skewness features are computed:

$$\text{FlowIAT}_{\text{skew}} = \begin{cases} \gamma_1(\text{FlowIAT}) & \text{if } |\text{FlowIAT}| > 2 \\ \text{NaN} & \text{otherwise} \end{cases} \quad (10)$$

$$\text{Size}_{\text{skew}} = \begin{cases} \gamma_1(S) & \text{if } |S| > 2 \\ \text{NaN} & \text{otherwise} \end{cases} \quad (11)$$

where $S = \{L(p) : p \in P_{\text{fwd}} \cup P_{\text{bwd}}\}$ is the set of all packet sizes. Positive skewness ($\gamma_1 > 0$) indicates right-tailed distributions with occasional large values, characteristic of traffic with intermittent delays or large object transfers. Near-zero skewness suggests symmetric, regular patterns typical of VoIP. Negative skewness reflects predominantly large values with occasional small packets, common in file transfer protocols.

Kurtosis Kurtosis measures the tailedness and peakedness of distributions:

$$\gamma_2(x) = \frac{\frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^4}{\left[\frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2\right]^2} - 3 \quad (12)$$

where the subtraction of 3 yields excess kurtosis relative to a normal distribution. Two kurtosis features characterize distribution shape:

$$\text{FlowIAT}_{\text{kurt}} = \begin{cases} \gamma_2(\text{FlowIAT}) & \text{if } |\text{FlowIAT}| > 3 \\ \text{NaN} & \text{otherwise} \end{cases} \quad (13)$$

$$\text{Packet_Size}_{\text{kurt}} = \begin{cases} \gamma_2(S) & \text{if } |S| > 3 \\ \text{NaN} & \text{otherwise} \end{cases} \quad (14)$$

High kurtosis ($\gamma_2 > 0$) indicates leptokurtic distributions with heavy tails and sharp peaks, suggesting protocol-driven applications with few distinct packet sizes. Near-zero kurtosis characterizes mesokurtic, normal-like distributions. Negative kurtosis reflects platykurtic, flat distributions with light tails, typical of encrypted or adaptive protocols with uniform size distributions.

Complete Feature Vector

The feature extraction process yields a **29-dimensional feature vector** for each flow, organized into complementary families:

Feature Family	Count	Features
Flow metadata	2	Duration, protocol
Directional volumetrics	4	N_{fwd} , N_{bwd} , $\bar{L}_{\text{fwd}}$, $\bar{L}_{\text{bwd}}$
Flow rates	2	Bytes/sec, packets/sec
Forward IAT statistics	6	Total, mean, median, min, max, std
Backward IAT statistics	6	Total, mean, median, min, max, std
Flow IAT statistics	5	Mean, median, min, max, std
Higher-order statistics	4	FlowIAT/Packet Size Skewness (2), kurtosis (2)
Total	29	

This multi-dimensional characterization captures complementary aspects of traffic behavior: volume features quantify overall activity level, temporal features encode timing patterns and regularity, size features reflect data transfer characteristics, shape features describe distribution properties, and directional features capture communication asymmetry.

Feature Normalization

To ensure numerical stability and comparable feature scales, feature-wise Z-score normalization was applied at train time:

$$z = \frac{x - \mu}{\sigma} \quad (15)$$

where μ and σ are the mean and standard deviation computed **exclusively from training data**. These statistics were then applied consistently to validation and test sets to prevent data leakage and ensure realistic evaluation of model generalization.

2.4 Feature Engineering

Feature Analysis

VoIP traffic is primarily characterized by **small, regular packets** exchanged **bidirectionally** at a **nearly constant rate**. Consequently, inter-arrival time statistics (FIAT/BIAT), packet size distributions, packet rate features, and forward-backward symmetry are the most discriminative features for distinguishing VoIP from other traffic types such as file transfer, streaming, or chat.

2D Feature Visualization: to gain an initial insight into the data distribution, we plotted the samples using two selected features: `std_flowiat` and `mean_flowiat`. Although the classes are not fully separable in this 2D space, we can observe that VoIP flows generally exhibit **lower mean and standard deviation of flow inter-arrival times**, reflecting their more **regular and consistent packet timing**. Additionally, the data show a reasonable degree of clustering for each class (see Figure 3).

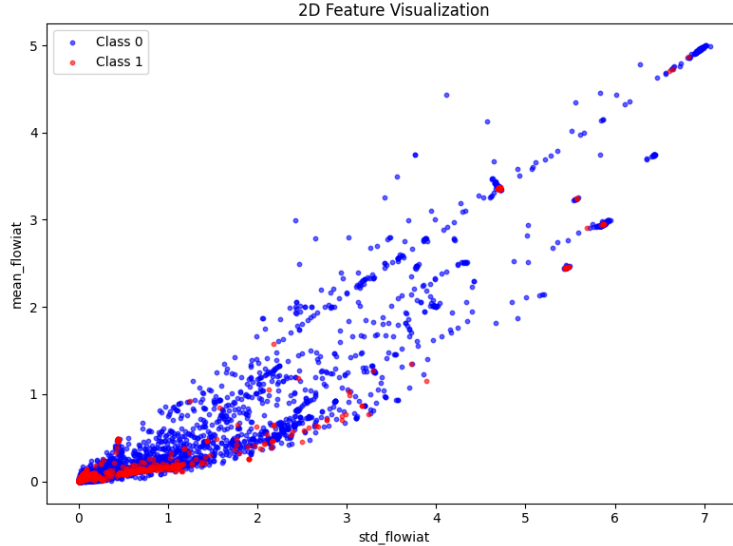


Figure 3: 2D scatter plot of flows using `std_flowiat` and `mean_flowiat`, color-coded by class. VoIP traffic shows lower values and clustered distribution.

t-SNE Visualization: we also applied t-SNE on all features and projected the resulting embeddings into 2D. The t-SNE visualization reveals the presence of several clusters, suggesting some degree of separability among different traffic types. Nevertheless, even in this

reduced space, VoIP flows **are not completely separable** from other traffic, highlighting the need for multi-dimensional analysis or supervised classification methods (see Figure 4).

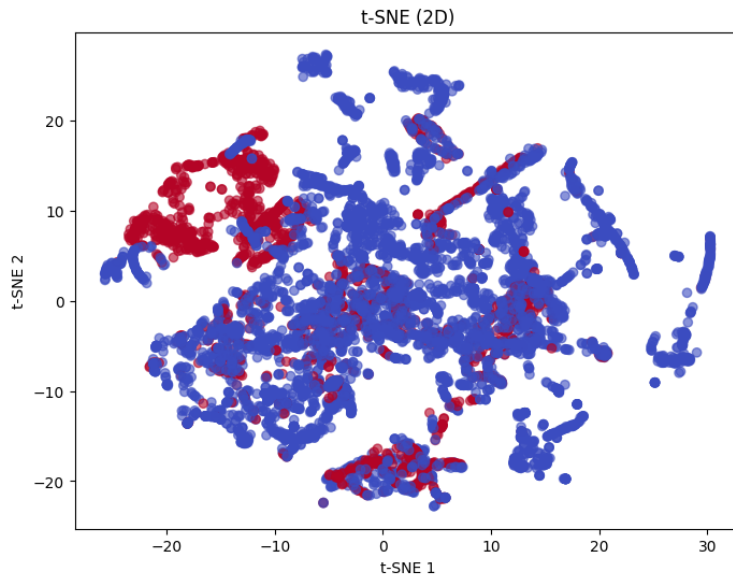


Figure 4: 2D t-SNE projection of all flow features. Some clustering patterns are visible, but classes are not fully separable.

Feature Reduction

High-dimensional feature spaces pose challenges for computational efficiency and model interpretability. Feature selection and dimensionality reduction techniques were employed to identify the most informative features and mitigate redundancy.

To address this, **Principal Component Analysis (PCA)** was employed as a dimensionality reduction technique while preserving most of the information content. Specifically, PCA was applied to project the original 29-dimensional feature space onto a lower-dimensional subspace spanned by orthogonal principal components ordered by decreasing explained variance. Components were retained to capture **95% of the total variance** in the training data, which in our case corresponded to **12 principal components**, ensuring that the majority of the information was preserved.

2.5 Class Imbalance Mitigation

Aggregating traffic classes into a binary classification problem resulted in a pronounced class imbalance, with VoIP traffic accounting for only 21.48% of the dataset traffic. Such imbalance is a common issue in network traffic classification and can severely degrade model performance, as learning algorithms tend to favor the majority class, leading to reduced recall and sensitivity for minority class instances.

To mitigate this effect, both data-level and algorithm-level imbalance handling strategies were investigated.

Synthetic Minority Oversampling Technique (SMOTE)

To address the pronounced class imbalance between VoIP and non-VoIP traffic, the Synthetic Minority Over-sampling Technique (SMOTE) was employed [4]. Unlike random over-sampling, which replicates minority samples and increases the risk of overfitting, SMOTE generates synthetic samples by interpolating between existing minority instances in the feature space.

For each minority sample, its k nearest neighbors are identified, and new samples are created along the line segments joining the sample with randomly selected neighbors. This strategy expands the decision region of the minority class and improves classifier generalization, particularly in high-dimensional feature spaces.

Synthetic Sample Generation Let $\mathbf{x}_i$ be a minority class sample and $\mathbf{x}_{nn}$ one of its k nearest minority neighbors. A synthetic sample $\mathbf{x}_{syn}$ is generated as:

$$\mathbf{x}_{syn} = \mathbf{x}_i + \lambda(\mathbf{x}_{nn} - \mathbf{x}_i), \quad (16)$$

where $\lambda \sim \mathcal{U}(0, 1)$ is a random number drawn from a uniform distribution.

The distance between samples is typically computed using the Euclidean metric:

$$d(\mathbf{x}_i, \mathbf{x}_j) = \sqrt{\sum_{m=1}^M (x_{im} - x_{jm})^2}, \quad (17)$$

where M denotes the feature dimensionality.

This interpolation mechanism forces the classifier to learn broader minority class regions instead of overfitting to isolated samples.

SMOTE with Edited Nearest Neighbors (SMOTEENN)

While SMOTE improves minority class representation, it may also introduce ambiguous or noisy synthetic samples, particularly near class boundaries. To address this limitation, the SMOTEENN algorithm combines oversampling with a subsequent cleaning step based on the **Edited Nearest Neighbors (ENN)** rule.

SMOTEENN operates in two stages: first, SMOTE is applied to generate synthetic minority samples. Subsequently, the ENN algorithm removes samples (from both majority and minority classes) whose class label differs from the majority label of their nearest neighbors (see Figure 5). This cleaning process helps eliminate noisy instances and overlapping samples near decision boundaries, resulting in a more compact and discriminative feature space.

Due to its ability to simultaneously balance the dataset and reduce class overlap, SMOTEENN was selected as the primary resampling strategy within the final experimental setup.

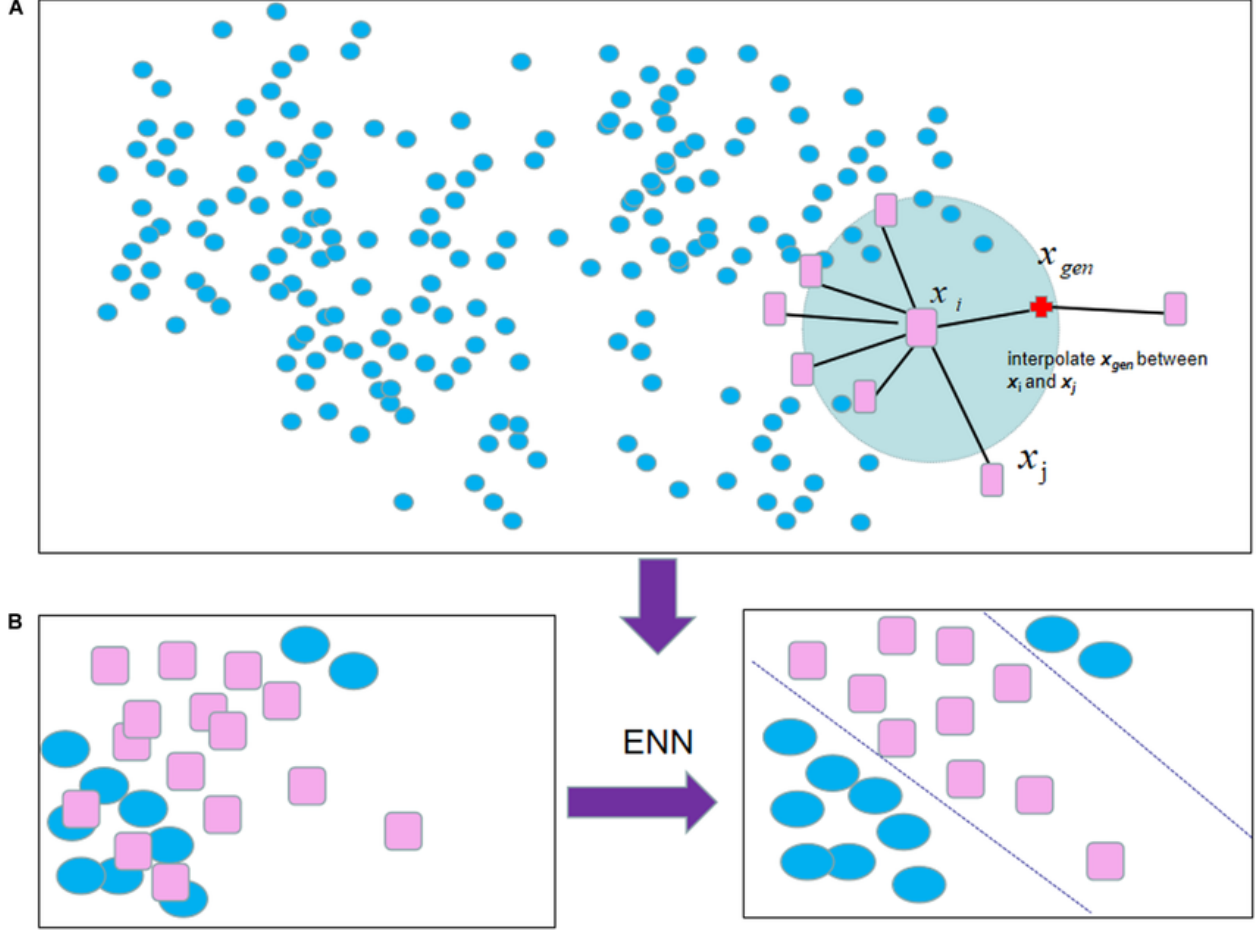


Figure 5: The SMOTE-ENN algorithm process: (A) SMOTE successively selects each minority class sample as a root instance for synthetic sample generation. (B) The Edited Nearest Neighbors (ENN) algorithm is subsequently applied to remove noisy and ambiguous samples introduced during the SMOTE oversampling stage. Source: [4]

Class-Weighted Training

In addition to data-level resampling, an algorithm-level imbalance mitigation strategy was employed through class-weighted training. This approach modifies the loss function by assigning higher penalties to misclassification errors affecting minority class samples.

By incorporating class-weighted loss, the classifier is encouraged to learn decision boundaries that are more sensitive to minority class patterns, without altering the underlying data distribution. The combination of synthetic oversampling and cost-sensitive learning provides a complementary strategy for addressing class imbalance in network traffic classification tasks.

Class weights were computed after SMOTE (if applied) as inversely proportional to the class frequencies, ensuring that errors on VoIP samples contribute more strongly to the optimization objective than errors on non-VoIP samples.

2.6 Model Selection and Hyperparameter Optimization

Three classifier families representing diverse learning paradigms were evaluated: Random Forests (ensemble learning), Support Vector Machines (kernel-based maximum margin), and Multi-Layer Perceptrons (neural network). Each classifier family offers distinct advantages and limitations for traffic classification tasks.

2.6.1 Random Forest Classifier

Random Forest is an ensemble method that aggregates predictions from multiple decision trees trained on bootstrap samples of the data. Each tree is constructed using a random subset of features at each split, introducing diversity that reduces overfitting and improves generalization. The final prediction is determined by majority voting across all trees.

Hyperparameter Space The following hyperparameters were optimized through grid search:

- **Number of Estimators (Es):** Number of decision trees in the ensemble. Tested values: $\{5, 10, 15\}$. Higher values generally improve performance but increase computational cost.
- **Max depth (Md):** Maximum depth of each tree, controlling model complexity. Tested range: $[3, 10]$. Deeper trees can capture complex decision boundaries but risk overfitting.
- **Minimum samples split (Mss):** Minimum fraction of samples required to split an internal node. Tested values: $\{0.05, 0.1, 0.15, 0.2\}$. Higher values prevent fragmentation and reduce overfitting.
- **Minimum samples leaf (Msl):** Minimum fraction of samples required in leaf nodes. Tested values: $\{0.05, 0.1\}$. Ensures leaves contain sufficient samples for reliable predictions.
- **Maximum samples (Ms):** Fraction of training data sampled for each tree. Tested range: $[0.3, 0.7]$. Controls bootstrap sampling intensity and tree diversity.
- **Maximum features (Mf):** Number of features considered at each split. Tested values: $\{\text{sqrt}, \log 2\}$. Feature subsampling introduces randomness and reduces correlation between trees.

2.6.2 Support Vector Machine Classifier

Support Vector Machines construct optimal hyperplanes that maximize the margin between classes in a high-dimensional feature space. The use of kernel functions enables SVMs to learn non-linear decision boundaries by implicitly mapping data to higher-dimensional spaces.

Hyperparameter Space Three critical hyperparameters were optimized:

- **Kernel:** Kernel function type determining the decision boundary shape. Tested kernels:
 - *Linear:* Suitable for linearly separable data, $K(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i^T \mathbf{x}_j$
 - *RBF (Radial Basis Function):* Handles non-linear relationships, $K(\mathbf{x}_i, \mathbf{x}_j) = \exp(-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2)$
 - *Polynomial:* Captures higher-order interactions, $K(\mathbf{x}_i, \mathbf{x}_j) = (\gamma \mathbf{x}_i^T \mathbf{x}_j + r)^d$
- **Regularization parameter (C):** Controls the trade-off between margin maximization and training error minimization. Tested values: $\{0.01, 1, 20\}$. Small C favors larger margins with greater tolerance for misclassification, while large C enforces stricter classification constraints.
- **Kernel coefficient (γ):** Defines the influence radius of individual training samples. Tested values: $\{0.001, 0.1, 1, \text{scale}\}$, where scale sets $\gamma = 1/(n_{\text{features}} \cdot \text{Var}(X))$. Small γ yields smooth decision boundaries, while large γ creates complex, localized boundaries.

2.6.3 Multi-Layer Perceptron Classifier

Multi-Layer Perceptrons are feedforward neural networks consisting of an input layer, one or more hidden layers with non-linear activation functions, and an output layer. MLPs can learn complex non-linear mappings through backpropagation training, making them suitable for capturing intricate traffic patterns.

Architecture Search Various network architectures were explored, with configurations ranging from shallow networks with few hidden layers to deeper architectures with up to eight hidden layers. Hidden layer sizes were progressively decreased to create funnel architectures that compress information toward the output layer.

Regularization Strategy To mitigate overfitting, dropout regularization was applied, randomly deactivating neurons during training to prevent co-adaptation. Tested dropout rates: $\{0.3, 0.5\}$. Additionally, early stopping was implemented to halt training when validation performance ceased improving, preventing unnecessary training iterations that could lead to overfitting.

Hyperparameter Space The primary hyperparameters optimized include:

- **Hidden layers (HL):** Tested network architecture configurations include **2 to 6 linear layers**, with **layer depths between 16 and 512 features**.
- **Dropout rate (DO):** Fraction of neurons randomly deactivated during training. Tested values: $\{0.3, 0.5\}$. Higher dropout rates provide stronger regularization but may impede learning if too aggressive.

3 Experimental Design

This section delineates the experimental framework employed to evaluate VoIP traffic discrimination performance. The experimental pipeline comprises four principal components: flow filtering configurations to assess the impact of traffic completeness, dataset splitting strategies to ensure realistic evaluation, model selection and hyperparameter optimization, and temporal aggregation analysis to examine the effect of time resolution on classification accuracy.

3.1 Flow Filtering Configurations

Flow completeness significantly influences feature extraction quality, particularly for **temporal features requiring sufficient packet observations**. To quantify this relationship, two progressively stringent filtering configurations were applied to assess the trade-off between dataset size and classification performance.

Configuration 1: Minimum One Packet Per Direction (Experiment 1)

This configuration enforces **bidirectional communication** by requiring at least one packet in both forward and backward directions. This ensures that directional asymmetry features can be meaningfully computed, filtering out unidirectional flows that lack the interactive characteristics typical of VoIP conversations.

- **Total flow count:** 16,616
- **VoIP flows:** 4,423 (26.09%)

Configuration 2: Minimum Two Packets Per Direction (Experiment 2)

A more stringent configuration requires **at least two packets in each direction**, ensuring sufficient observations for robust computation of directional inter-arrival time statistics. This selects flows exhibiting sustained bidirectional communication, characteristic of interactive real-time applications.

- **Total flow count:** 7,277
- **VoIP flows:** 1,942 (26.74%)

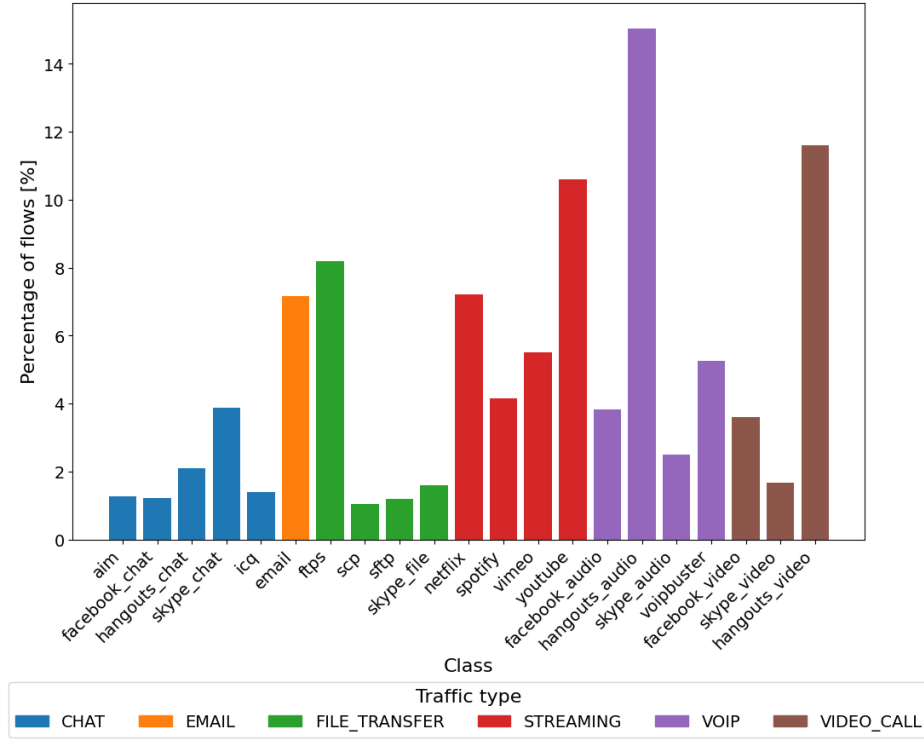


Figure 6: Class distribution for Configuration 1: Minimum one packet per direction.

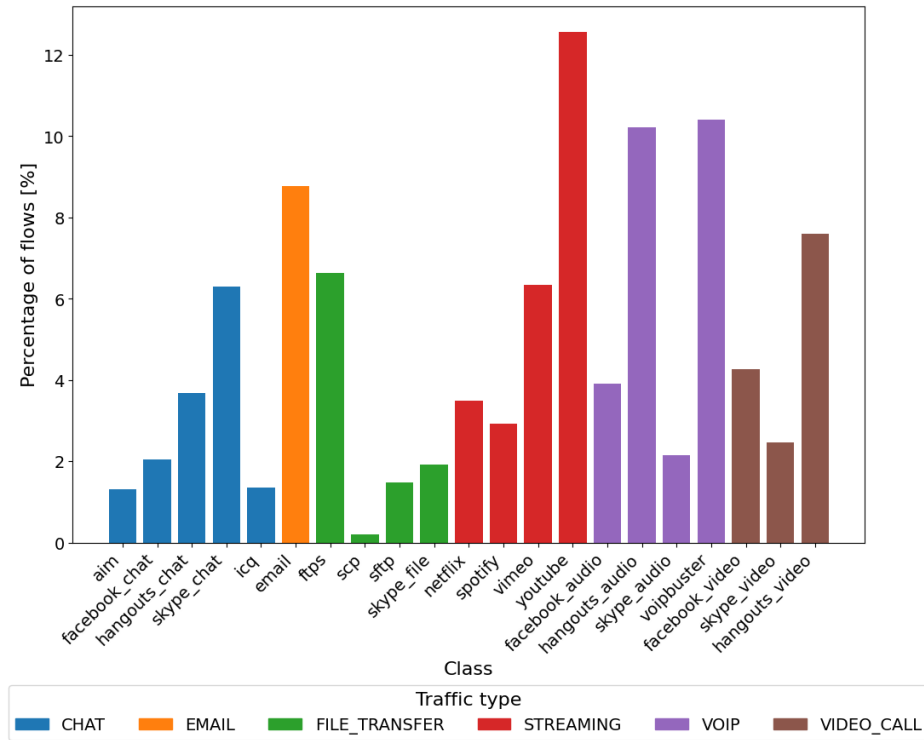


Figure 7: Class distribution for Configuration 2: Minimum two packets per direction.

Filtering Strategy Analysis

This progressive filtering results reveal a clear trade-off between dataset size and classification performance. Each successive filtering stage removes incomplete flows that introduce feature noise, thereby improving the signal quality of the remaining flows. Notably, the VoIP class proportion increases from 21.02% to 26.74% as filtering stringency increases, suggesting that VoIP flows are more likely to exhibit sustained bidirectional communication compared to certain non-VoIP traffic types (e.g., DNS queries, failed connection attempts, or unidirectional streaming).

3.2 Data Split

To rigorously evaluate model performance, the dataset was partitioned using **stratified 4-fold cross-validation**, stratified by all traffic labels and grouped by *flow ID*. VOIP-type traffic (e.g., `skype_audio`, `hangouts_audio`) was annotated as the **label of interest** (1), while all other traffic types were labeled as 0. This strategy ensured three important considerations:

1. **Balanced representation:** All traffic types are present in both training and validation sets. By annotating VOIP/Rest as the target after forming folds, we prevent scenarios in which some classes might be missing in the training set, which could lead to misclassification at test time.
2. **Flow-wise separation:** Traffic from a single flow is never split across training and validation folds, avoiding leakage of highly similar patterns between the sets and ensuring a realistic evaluation of generalization.
3. **Robust evaluation under imbalance:** Stratified cross-validation provides a comprehensive measure of model capability across all folds, especially in the presence of unbalanced datasets, capturing the model’s ability to correctly classify both majority and minority classes.

Figure 8 shows the class distribution across the four folds for the 15-second bidirectional flows setup. It can be observed that stratified group splitting succeeds in maintaining a nearly balanced distribution across folds, despite the dataset containing a high number of flows per flow ID.

4 Results and Analysis

This section presents comprehensive experimental results across flow filtering configurations, classifier types, and temporal aggregation windows. Performance is evaluated using accuracy, F1-score, precision, and recall on held-out test sets, with particular emphasis on F1-score as the primary metric due to class imbalance considerations.

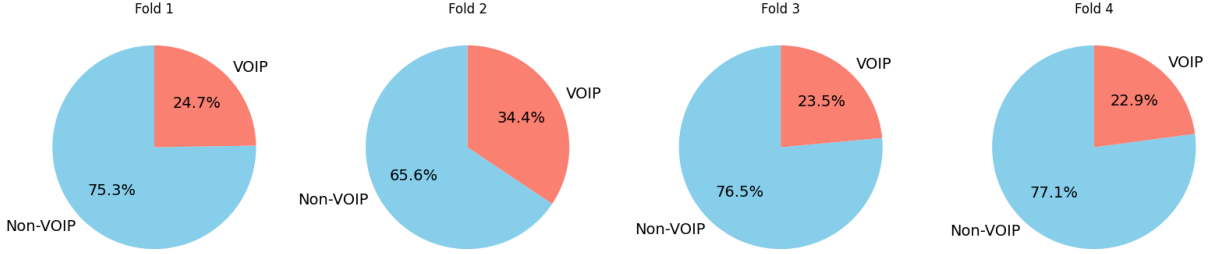


Figure 8: Class distribution across the four folds in the 15-second bidirectional flows setup.

1. Experiment 1 - Minimum One Packet per Direction

Experiment 1 evaluates a relaxed flow filtering strategy in which bidirectional communication is enforced by requiring only a single packet in each direction. This setting results in noisier flows that contain weaker interaction patterns, which is reflected in the overall moderate performance observed across all classifiers. Among the evaluated models, SVM consistently achieves the highest performance, reaching an F1-score of 64.91% at a 15-second aggregation window without additional preprocessing. This indicates that margin-based classifiers are better suited for handling noisy and weakly structured data.

MLP performance in this experiment is highly sensitive to preprocessing choices. While baseline configurations underperform, the combination of SMOTE and PCA significantly improves short-window results, yielding an F1-score of up to 70.02%. However, large discrepancies between training and test performance suggest limited generalization. Random Forest exhibits the weakest performance overall, with F1-scores rarely exceeding 58%, indicating a limited ability to model discriminative patterns under minimal bidirectional constraints. Overall, Experiment 1 favors robust classifiers and minimal preprocessing, trading peak performance for increased temporal stability.

2. Experiment 2 - Minimum Two Packets per Direction

Experiment 2 enforces stricter bidirectional filtering by requiring at least two packets in each direction, resulting in cleaner and more informative flows. This improvement in data quality leads to substantial performance gains across all classifiers, particularly at short aggregation windows. MLP achieves the best overall performance in this setting, reaching 82.82% accuracy and a 76.61% F1-score at 15 seconds when combined with SMOTE and PCA.

SVM also benefits significantly from the stricter filtering, achieving an F1-score of 76.23% without synthetic augmentation, which suggests that well-defined decision boundaries emerge naturally in clean data. While the performance of Random Forest improves compared to Experiment 1, it remains more susceptible to overfitting, especially when synthetic sampling is applied. Despite higher peak performance, Experiment 2 introduces sensitivity to temporal aggregation, with performance degrading rapidly as the flow duration increases.

Table 1: Classification performance for Experiment 1 across different flow durations for RF. The best result is shown in **bold**, the second-best is underlined, and the third-best is in **red**.

Model	Aggr.	Configuration	PCA	SMOTE	Split	Acc (%)	F1 (%)	Prec (%)	Rec (%)
RF	15s	Es=15, Mss=0.05, Md=4, Msl=0.05, Ms=0.7, Mf=sqrt	No	No	Train Test	73.93 73.74	59.65 <u>57.39</u>	50.63 48.68	72.26 70.16
		Es=15, Mss=0.1, Md=4, Msl=0.05, Ms=0.5, Mf=sqrt	No	Yes	Train Test	82.64 72.63	82.21 55.62	85.21 48.16	79.45 66.10
		Es=10, Mss=0.05, Md=5, Msl=0.05, Ms=0.7, Mf=sqrt	Yes	No	Train Test	73.96 73.74	59.79 57.99	50.77 49.90	72.86 <u>69.64</u>
		Es=15, Mss=0.15, Md=4, Msl=0.05, Ms=0.7, Mf=sqrt	Yes	Yes	Train Test	82.57 <u>73.16</u>	81.77 55.95	86.57 <u>49.00</u>	77.51 65.78
		Es=15, Mss=0.05, Md=4, Msl=0.05, Ms=0.7, Mf=log2	No	No	Train Test	67.98 67.62	54.85 54.07	45.06 44.58	70.09 68.74
		Es=15, Mss=0.05, Md=5, Msl=0.05, Ms=0.7, Mf=sqrt	No	Yes	Train Test	80.60 68.97	78.93 <u>55.70</u>	83.58 <u>46.11</u>	74.79 <u>70.38</u>
		Es=15, Mss=0.05, Md=5, Msl=0.05, Ms=0.7, Mf=sqrt	Yes	No	Train Test	69.53 69.15	56.67 56.06	46.80 46.35	71.82 71.00
		Es=15, Mss=0.05, Md=4, Msl=0.05, Ms=0.7, Mf=sqrt	Yes	Yes	Train Test	80.44 <u>69.00</u>	78.74 <u>55.68</u>	83.46 <u>46.13</u>	74.53 70.26
	120s	Es=15, Mss=0.15, Md=4, Msl=0.05, Ms=0.7, Mf=log2	No	No	Train Test	65.38 65.44	55.86 <u>55.89</u>	44.56 <u>44.61</u>	74.87 74.84
		Es=15, Mss=0.05, Md=5, Msl=0.05, Ms=0.7, Mf=sqrt	No	Yes	Train Test	82.94 79.09	82.50 48.59	87.31 40.10	78.29 63.65
		Es=15, Mss=0.05, Md=4, Msl=0.05, Ms=0.7, Mf=sqrt	Yes	No	Train Test	79.07 <u>65.97</u>	77.57 56.24	81.72 45.10	73.89 <u>74.72</u>
		Es=10, Mss=0.05, Md=4, Msl=0.05, Ms=0.7, Mf=sqrt	Yes	Yes	Train Test	81.43 79.10	80.66 47.69	86.89 39.19	75.30 62.16
	600s	Kernel=RBF, C=20, γ =scale	No	No	Train Test	84.55 76.80	83.93 64.91	85.22 53.98	82.67 <u>81.37</u>
		Kernel=RBF, C=20, γ =scale	No	Yes	Train Test	86.63 <u>73.83</u>	86.52 <u>58.86</u>	87.95 <u>50.17</u>	85.15 71.89
		Kernel=RBF, C=20, γ =scale	Yes	No	Train Test	76.59 66.18	78.18 56.97	71.70 42.86	85.95 84.92
		Kernel=RBF, C=20, γ =scale	Yes	Yes	Train Test	80.36 66.33	81.75 53.66	76.94 42.29	87.19 74.44
SVM	15s	Kernel=RBF, C=20, γ =scale	No	No	Train Test	70.60 <u>69.10</u>	60.38 <u>57.99</u>	48.22 <u>46.58</u>	80.72 76.93
		Kernel=RBF, C=100, γ =scale	No	Yes	Train Test	82.57 64.98	83.04 56.05	78.84 43.02	87.79 <u>80.56</u>
		Kernel=RBF, C=20, γ =scale	Yes	No	Train Test	70.36 69.25	60.04 58.05	47.97 46.71	80.23 76.77
		Kernel=RBF, C=0.1, γ =scale	Yes	Yes	Train Test	74.39 57.73	77.02 52.81	68.27 38.26	88.37 85.37
	120s	Kernel=RBF, C=20, γ =scale	No	No	Train Test	68.72 67.82	61.10 59.86	48.03 47.14	83.95 <u>81.99</u>
		Kernel=RBF, C=100, γ =scale	No	Yes	Train Test	80.79 64.79	80.54 56.83	79.80 44.36	81.37 79.20
		Kernel=RBF, C=20, γ =scale	Yes	No	Train Test	68.61 <u>67.73</u>	60.56 <u>59.26</u>	47.89 <u>46.99</u>	82.38 80.22
		Kernel=RBF, C=100, γ =scale	Yes	Yes	Train Test	72.41 56.48	75.85 53.65	68.21 38.97	85.44 86.09
	600s	Kernel=RBF, C=20, γ =scale	No	No	Train Test	68.72 67.82	61.10 59.86	48.03 47.14	83.95 <u>81.99</u>
		Kernel=RBF, C=100, γ =scale	No	Yes	Train Test	80.79 64.79	80.54 56.83	79.80 44.36	81.37 79.20
		Kernel=RBF, C=20, γ =scale	Yes	No	Train Test	68.61 <u>67.73</u>	60.56 <u>59.26</u>	47.89 <u>46.99</u>	82.38 80.22
		Kernel=RBF, C=100, γ =scale	Yes	Yes	Train Test	72.41 56.48	75.85 53.65	68.21 38.97	85.44 86.09

Table 2: Classification performance for Experiment 1 across different flow durations for MLP. The best result is shown in **bold**, the second-best is underlined, and the third-best is in **red**.

Model	Size	Configuration	PCA	SMOTE	Split	Acc (%)	F1 (%)	Prec (%)	Rec (%)
MLP	15s	[128,128,128,64,32,16], DO=0.3	No	No	Train	71.40	69.68	77.28	71.48
					Test	58.51	57.30	65.29	67.86
		<u>[128,128,64,32], DO=0.3</u>	<u>No</u>	<u>Yes</u>	Train	90.05	90.05	90.08	90.04
					Test	<u>72.62</u>	<u>69.20</u>	<u>69.14</u>	73.88
		[256,256,128,64], DO=0.3	Yes	No	Train	70.97	68.93	77.93	71.06
					Test	59.35	58.12	65.96	68.67
		[32,32,16], DO=0.3	Yes	Yes	Train	87.07	87.06	87.20	87.08
					Test	73.99	70.02	69.49	<u>73.73</u>
		[256,128,64,32], DO=0.3	No	No	Train	72.15	70.42	78.94	72.15
					Test	59.98	59.59	66.97	69.67
	120s	[256,128,64], DO=0.3	No	Yes	Train	88.21	88.15	88.96	88.20
					Test	66.58	65.43	68.60	73.05
		[128,64], DO=0.3	Yes	No	Train	70.27	68.22	77.52	70.38
					Test	57.66	57.45	66.25	68.25
	600s	<u>[32,32,16], DO=0.3</u>	<u>Yes</u>	<u>Yes</u>	Train	84.40	84.39	84.58	84.43
					Test	<u>66.40</u>	<u>64.84</u>	<u>66.98</u>	<u>71.19</u>
		[256,128,64], DO=0.3	No	No	Train	71.07	69.52	76.55	71.11
					Test	61.45	61.18	67.90	70.19
		[128,64], DO=0.3	No	Yes	Train	86.41	86.37	86.70	86.38
					Test	65.98	65.12	68.49	71.99
		[256,128,64], DO=0.3	Yes	No	Train	70.43	68.27	77.29	70.17
					Test	58.74	58.66	67.92	69.09
		<u>[128,64], DO=0.3</u>	<u>Yes</u>	<u>Yes</u>	Train	85.86	85.83	86.25	85.91
					Test	<u>65.66</u>	<u>64.79</u>	<u>68.21</u>	<u>71.62</u>

3. Experiment 1 vs. Experiment 2

A clear quality-coverage trade-off is observed between the two experimental settings. Experiment 2 consistently outperforms Experiment 1 by approximately 10-15% across all evaluation metrics at shorter flows constraints, confirming that **stricter bidirectional constraints enhance signal quality**. However, this improvement comes at the cost of temporal robustness, as performance in Experiment 2 degrades sharply for longer windows. In contrast, Experiment 1 shows a more gradual degradation curve, indicating greater tolerance to maximum flow duration but lower absolute performance. Consequently, Experiment 2 is better suited for early traffic classification, while Experiment 1 is preferable when longer observation periods are required.

4. Maximum Flow Size Impact

Flow duration has a pronounced impact on classification performance, particularly under strict filtering conditions. In Experiment 2, discriminative performance collapses rapidly as flow duration increases, with F1-scores dropping from approximately 80% at 15 seconds to 40-55% at 600 seconds. This trend suggests that the most informative features are concentrated in the initial packet exchanges of a flow. Experiment 1 exhibits smoother degradation

Table 3: Classification performance for Experiment 2 across different flow durations for RF. The best result is shown in **bold**, the second-best is underlined, and the third-best is in **red**.

Model	Size	Configuration	PCA	SMOTE	Split	Acc (%)	F1 (%)	Prec (%)	Rec (%)
RF	15s	Es=15, Mss=0.05, Md=4, Msl=0.05, Ms=0.7, Mf=sqrt	No	No	Train Test	73.93 72.91	59.65 57.39	50.63 48.68	72.26 70.16
		Es=15, Mss=0.2, Md=3, Msl=0.05, Ms=0.6, Mf=sqrt	No	Yes	Train Test	84.85 81.56	84.16 61.24	91.57 63.04	77.87 62.59
		Es=15, Mss=0.05, Md=3, Msl=0.05, Ms=0.7, Mf=sqrt	Yes	No	Train Test	82.53 <u>79.69</u>	68.85 <u>60.22</u>	65.00 <u>59.00</u>	73.55 64.40
		Es=5, Mss=0.05, Md=3, Msl=0.05, Ms=0.5, Mf=sqrt	Yes	Yes	Train Test	82.86 78.89	82.54 59.51	87.18 55.71	78.39 <u>65.08</u>
	120s	Es=15, Mss=0.05, Md=4, Msl=0.05, Ms=0.7, Mf=log2	No	No	Train Test	67.98 67.62	54.85 54.07	45.06 44.58	70.09 68.74
		Es=15, Mss=0.05, Md=5, Msl=0.05, Ms=0.7, Mf=sqrt	No	Yes	Train Test	82.94 <u>79.10</u>	80.66 47.69	86.89 39.19	75.30 62.16
		Es=15, Mss=0.05, Md=4, Msl=0.05, Ms=0.7, Mf=sqrt	Yes	No	Train Test	77.04 75.61	48.86 45.66	37.40 36.08	70.51 <u>64.52</u>
		Es=15, Mss=0.05, Md=4, Msl=0.05, Ms=0.7, Mf=log2	Yes	Yes	Train Test	82.40 79.15	81.77 <u>48.12</u>	87.55 <u>39.50</u>	76.75 62.79
	600s	Es=15, Mss=0.15, Md=4, Msl=0.05, Ms=0.7, Mf=log2	No	No	Train Test	65.38 65.44	55.86 55.89	44.56 44.61	74.87 <u>74.84</u>
		Es=10, Mss=0.05, Md=4, Msl=0.05, Ms=0.7, Mf=log2	No	Yes	Train Test	79.38 75.24	79.38 <u>42.55</u>	82.53 <u>30.91</u>	76.49 68.42
		Es=15, Mss=0.05, Md=5, Msl=0.05, Ms=0.7, Mf=log2	Yes	No	Train Test	72.00 70.91	43.16 41.01	29.69 28.28	78.90 74.98
		Es=15, Mss=0.05, Md=4, Msl=0.05, Ms=0.7, Mf=sqrt	Yes	Yes	Train Test	79.36 <u>73.07</u>	79.99 41.01	80.44 29.08	79.60 69.95
SVM	15s	Kernel=RBF, C=20, γ =scale	No	No	Train Test	89.24 86.39	89.51 <u>80.43</u>	91.52 <u>82.79</u>	87.58 <u>78.20</u>
		Kernel=poly, C=100, γ =scale	No	Yes	Train Test	90.70 <u>82.60</u>	90.82 67.91	93.06 62.28	88.71 77.46
		Kernel=RBF, C=20, γ =scale	Yes	No	Train Test	84.98 82.23	85.55 76.23	86.24 73.09	84.88 79.65
		Kernel=RBF, C=100, γ =scale	Yes	Yes	Train Test	87.10 79.13	87.62 63.67	87.17 55.52	88.09 76.74
	120s	Kernel=RBF, C=20, γ =scale	No	No	Train Test	76.71 73.82	52.87 45.09	39.15 <u>34.33</u>	82.68 67.58
		Kernel=linear, C=50, γ =scale	No	Yes	Train Test	76.35 <u>76.05</u>	74.90 43.48	82.48 <u>34.24</u>	68.79 59.93
		Kernel=RBF, C=20, γ =scale	Yes	No	Train Test	76.49 73.60	50.84 <u>43.72</u>	38.43 33.73	76.82 <u>65.22</u>
		Kernel=linear, C=0.01, γ =scale	Yes	Yes	Train Test	69.65 82.45	62.55 42.84	86.08 43.48	49.19 42.92
	600s	Kernel=RBF, C=20, γ =scale	No	No	Train Test	82.77 81.11	54.37 <u>47.40</u>	42.35 <u>37.90</u>	76.30 <u>63.64</u>
		Kernel=RBF, C=100, γ =scale	No	Yes	Train Test	80.79 64.78	80.54 56.83	79.80 44.36	81.37 79.20
		Kernel=RBF, C=20, γ =scale	Yes (0.95)	No	Train Test	80.64 <u>78.90</u>	50.60 43.85	38.51 34.05	73.83 61.69
		Kernel=RBF, C=100, γ =scale	Yes (0.9)	Yes	Train Test	80.67 77.36	79.82 41.03	86.95 31.59	73.78 58.74

Table 4: Classification performance for Experiment 2 across different flow durations for MLP. The best result is shown in **bold**, the second-best is underlined, and the third-best is in **red**.

Model	Size	Configuration	PCA	SMOTE	Split	Acc (%)	F1 (%)	Prec (%)	Rec (%)
MLP	15s	[256,128,64], DO=0.3	No	No	Train	81.71	81.19	85.45	81.67
					Test	69.81	66.45	68.74	76.04
		<u>[64,64,32], DO=0.3</u>	<u>No</u>	<u>Yes</u>	Train	93.49	93.48	93.54	93.48
					Test	<u>80.53</u>	<u>74.14</u>	<u>72.79</u>	<u>77.23</u>
		[512,256,128], DO=0.3	Yes	No	Train	81.57	80.95	85.26	81.48
					Test	71.99	68.01	70.74	77.21
		[64,32], DO=0.3	Yes	Yes	Train	90.91	90.90	90.99	90.91
					Test	82.82	76.61	75.35	78.91
	120s	[512,256,128], DO=0.3	No	No	Train	76.74	75.48	83.64	76.73
					Test	55.64	51.42	59.27	67.30
		[128,64,32,16,8], DO=0.3	No	Yes	Train	90.76	90.75	90.88	90.75
					Test	75.71	64.84	63.70	71.12
		[512,256,128], DO=0.3	Yes	No	Train	72.85	70.66	81.82	72.86
					Test	51.58	48.51	58.94	66.15
		<u>[128,64], DO=0.3</u>	<u>Yes</u>	<u>Yes</u>	Train	88.87	88.86	88.94	88.88
					Test	<u>73.03</u>	<u>62.56</u>	<u>62.13</u>	<u>69.63</u>
	600s	[512,256,128], DO=0.3	No	No	Train	79.58	78.75	85.08	79.63
					Test	62.53	55.64	60.54	72.75
		[512,256,128], DO=0.3	No	Yes	Train	95.73	95.73	95.74	95.72
					Test	80.12	68.82	66.52	77.70
		[512,256,128], DO=0.3	Yes	No	Train	74.62	73.15	81.91	74.78
					Test	53.42	48.93	58.86	68.62
		<u>[256,128,64,32], DO=0.3</u>	<u>Yes</u>	<u>Yes</u>	Train	91.36	91.36	91.39	91.37
					<u>Test</u>	<u>77.39</u>	<u>66.39</u>	<u>64.90</u>	<u>76.62</u>

with increasing maximum flow duration, implying that noise dominates feature distributions and masks temporal sensitivity. Among the models, SVM demonstrates the most stable degradation behavior, while MLP is more sensitive to flow duration without proper regularization.

5. SMOTEENN Usage

The effectiveness of synthetic sampling techniques is strongly dependent on data quality and classifier choice. In Experiment 1, synthetic augmentation generally degrades performance across all models, particularly for Random Forest and SVM, where it amplifies noise and leads to overfitting. In both experiments SMOTE provides **substantial benefits only for MLP**, improving F1-scores by up to 10-12% at short flow durations. For SVM and Random Forest, synthetic samples introduce ambiguity into otherwise clean decision boundaries, resulting in performance degradation. These findings indicate that synthetic sampling should be applied selectively and primarily in conjunction with neural models and high-quality filtered data.

Table 5: Best performing configurations for Experiment 1. The best result is shown in **bold**, the second-best is underlined, and the third-best is in **red**.

Model	Size	Configuration	PCA	SMOTE	Acc (%)	F1 (%)	Prec (%)	Rec (%)
RF	15s	Es=15, Mss=0.05, Md=4, Msl=0.05	Yes	No	73.74	57.99	49.90	69.64
SVM	15s	Kernel=RBF, C=20, γ =scale	No	No	76.80	<u>64.91</u>	<u>53.98</u>	81.37
MLP	15s	[32,32,16], DO=0.3	Yes	Yes	<u>73.99</u>	70.02	69.49	<u>73.73</u>

Table 6: Best performing configurations for Experiment 2. The best result is shown in **bold**, the second-best is underlined, and the third-best is in **red**.

Model	Win.	Configuration	PCA	SMOTE	Acc (%)	F1 (%)	Prec (%)	Rec (%)
RF	15s	Es=15, Mss=0.2, Md=3, Msl=0.05	No	Yes	81.56	61.24	63.04	62.59
SVM	15s	Kernel=RBF, C=20, γ =scale	Yes	No	82.23	76.23	73.09	<u>79.65</u>
MLP	15s	[64,32], DO=0.3	Yes	Yes	82.82	76.61	75.35	78.91

6. Feature Reduction

Feature reduction via PCA demonstrates limited and model-specific benefits. For SVM and Random Forest, PCA consistently removes discriminative information, leading to noticeable performance degradation in both experiments. In contrast, **MLP benefits from PCA** when combined with **synthetic sampling**, where dimensionality reduction acts as a regularization mechanism that mitigates noise introduced by artificial samples, while over-sampling helps re-balance the class distribution.

7. Feature Importance Analysis

To gain insight into which flow characteristics contribute most to distinguishing VoIP traffic, we employed the **internal feature importance mechanism of the Random Forest classifier**. Random Forest evaluates feature relevance based on the average decrease in impurity (Gini importance) across all trees in the ensemble, providing a quantitative measure of each feature’s influence on model decisions.

For each cross-validation fold and hyperparameter configuration, feature importance was extracted and aggregated across folds and experiments. This approach allowed us to identify which features most consistently contributed to classification performance, even under variations in training data and model hyper-parameters.

The overall top three features identified across experiments were:

- **mean_flowiat**: Average inter-arrival time across the entire flow, reflecting the regularity of packet transmissions typical of VoIP codecs.
- **max_flowiat**: Maximum inter-arrival time, capturing burstiness and occasional pauses in traffic.
- **flowBytesPerSecond**: Flow-level throughput, distinguishing low-rate VoIP from higher-rate streaming or file transfers.

These results confirm that temporal features (inter-arrival times) and volumetric features (bytes per second) are determinants in identifying interactive real-time traffic.

8. Comparison with ISCX2016 ARFF

9. Overall Comparative Conclusion

Overall, stricter bidirectional filtering yields superior performance by enhancing flow-level signal quality, but introduces strong sensitivity to maximum flow duration. Relaxed filtering offers greater temporal robustness at the cost of reduced accuracy. Among the evaluated classifiers, MLP demonstrates the highest adaptability when paired with appropriate pre-processing, SVM performs best on clean data, with minimal intervention, and Random Forest shows limited robustness under synthetic augmentation (Table 5). The observed performance trends, together with feature importance analysis, confirm that early temporal dynamics dominate discriminative power. In practical deployments, strict filtering should be preferred for early traffic identification, while relaxed filtering is more suitable for scenarios requiring extended observation windows.

5 Conclusion

This work demonstrates that flow filtering strictness and maximum flow duration are the primary factors governing VoIP classification performance. Enforcing stricter bidirectional constraints significantly improves data quality and yields consistent gains of 10-15% in F1-score for short flows. This confirms that complete bidirectional interactions are essential for reliable discrimination. However, these gains diminish rapidly as the maximum flow duration increases.

Across all classifiers, peak performance is achieved for shorter flows (15 seconds), highlighting that VoIP traffic is best characterized by early temporal dynamics. As the maximum allowed flow duration increases, discriminative information rapidly diminishes, particularly under strict filtering. Relaxed filtering produces lower peak accuracy but offers smoother degradation and greater temporal stability.

Classifier behavior is strongly model-dependent. SVM provides robust and stable performance on clean data with minimal pre-processing, while MLP achieves the highest peak performance when combined with SMOTE and PCA (Table 6), but remains sensitive to overfitting and temporal aggregation. Random Forest shows limited robustness under synthetic augmentation, restricting its effectiveness in heavily preprocessed settings.

Pre-processing techniques must be applied selectively. Synthetic sampling and PCA benefit only neural models, where they act as effective regularizers, while consistently degrading SVM and Random Forest performance. Feature importance analysis confirms that inter-arrival time statistics dominate discriminative power, explaining both the effectiveness of short flow duration analysis and the observed temporal sensitivity.

Overall, the results indicate that early flow observation, strict filtering, and classifier-specific pre-processing are key to effective VoIP traffic classification, offering practical guidance for real-world deployment scenarios.

Repository and Code

The source code and additional materials can be found on GitHub:
`network-traffic-classifier`

References

- [1] A. Habibi Lashkari, G. Draper Gil, M. S. I. Mamun, and A. A. Ghorbani, "Characterization of Encrypted and VPN Traffic Using Time-Related Features," in *Proceedings of the 2nd International Conference on Information Systems Security and Privacy (ICISSP)*, 2016, pp. 407–414.
- [2] Mr-Pepe, "iscx-analysis/iscx-analysis at master · Mr-Pepe/iscx-analysis," GitHub, 2025. <https://github.com/Mr-Pepe/iscx-analysis/tree/master/iscx-analysis> (accessed Dec. 30, 2025).
- [3] A. Chaudhary, S. R. Kolhe, and R. Kamal, "SMOTE for Handling Imbalanced Data Problem: A Review," *International Journal of Computer Science and Information Technologies*, vol. 7, no. 4, pp. 1597–1601, 2016.
- [4] Q. Gao et al., "Identification of Orphan Genes in Unbalanced Datasets Based on Ensemble Learning," *Frontiers in Genetics*, vol. 11, Oct. 2020.